E-COMMERCE DATA ANALYSIS USING MYSQL: SQL CLAUSES AND OPERATORS🛒

Database: ECommerceDB

Technology: MySQL
Project Type: SQL Practice / Coding Challenge

1. Project Overview

This project focuses on practicing fundamental **SQL clauses and operators** using a simple e-commerce database.

The database, **ECommerceDB**, contains two relational tables: Product and Sales. Sample product and sales data were created and analyzed using different SQL clauses and operators.

The project demonstrates how SQL can be used to filter, compare, calculate, search, and retrieve specific information from relational databases.

2. Project Objectives

The main objectives of this project are to:

- Create an e-commerce database using MySQL.
- Create relational Product and Sales tables.
- Define primary keys and foreign key relationships.
- Insert sample product and sales data.
- Retrieve unique values using DISTINCT.
- Rename columns using aliases with AS.
- Filter records using the WHERE clause.
- Apply comparison operators.

- Perform calculations using arithmetic operators.
- Combine conditions using logical operators.
- Handle NULL values using IS NULL and IS NOT NULL.
- Filter records using IN and NOT IN.
- Filter ranges using BETWEEN and NOT BETWEEN.
- Search text patterns using LIKE and NOT LIKE.
- Combine multiple conditions in exam-level SQL queries.

3. Database Design

The project uses a database named:

```
CREATE DATABASE ECommerceDB;

USE ECommerceDB;
```

The database contains two tables:

Product Table

The Product table stores information about products available in the e-commerce system.

Column	Data Type	Description
product_id	INT	Unique product identifier
product_name	VARCHAR(100)	Name of the product
price	DECIMAL(10,2)	Product price

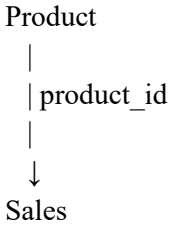
Sales Table

The Sales table stores product sales transactions.

Column	Data Type	Description
sale_id	INT	Unique sales transaction ID
product_id	INT	Product identifier
quantity	INT	Quantity sold
sale_amount	DECIMAL(10,2)	Total sales amount

4. Table Relationships

The Product and Sales tables are connected through the product_id column.



- Product.product_id is the **Primary Key**.
- Sales.sale_id is the **Primary Key**.
- Sales.product_id is a **Foreign Key** referencing Product.product_id.

This relationship allows sales transactions to be associated with the corresponding products.

5. Table Creation

Product Table

```
CREATE TABLE Product (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) CHECK (price > 0)
);
```

Sales Table

```
CREATE TABLE Sales (
    sale_id INT PRIMARY KEY,
    product_id INT,
    quantity INT CHECK (quantity > 0),
    sale_amount DECIMAL(10,2) CHECK (sale_amount > 0),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
);
```

6. Sample Data

Product Data

The following products were inserted:

Product ID	Product Name	Price
1	Laptop	85,000
2	Smartphone	45,000
3	Headphones	5,000
4	Keyboard	1,200
5	Mouse	800
6	Monitor	15,000
7	Webcam	3,500

Sales Data

Sale ID	Product ID	Quantity	Sale Amount
1	1	2	170,000
2	2	3	135,000
3	3	5	25,000
4	4	10	12,000
5	5	15	12,000
6	6	2	30,000
7	7	4	14,000

7. SQL Clauses and Operators Practiced

7.1 DISTINCT

DISTINCT is used to return unique values.

Unique Product Names

```
SELECT DISTINCT product_name
FROM Product;
```

Unique Product IDs

```
SELECT DISTINCT product_id
FROM Sales;
```

Unique Sale Quantities

```
SELECT DISTINCT quantity
FROM Sales;
```

8. Column Aliasing Using AS

The AS keyword is used to provide a temporary name or alias to a column.

```
SELECT product_name AS Product_Name
FROM Product;
```

Another example:

```
SELECT price AS Product_Price
FROM Product;
```

9. WHERE Clause

The WHERE clause filters records based on a specified condition.

Products Above 10,000

```
SELECT price
FROM Product
WHERE price > 10000;
```

Products Below 5,000

```
SELECT price
FROM Product
WHERE price < 5000;
```

Sales with Quantity Equal to 2

```
SELECT quantity
FROM Sales
WHERE quantity = 2;
```

10. Comparison Operators

Comparison operators are used to compare values.

Common operators include:

Operator	Meaning
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
=	Equal to
<> / !=	Not equal to

Products Priced at 15,000 or More

```
SELECT price
FROM Product
WHERE price >= 15000;
```

Sales Where Quantity Is Not 5

```
SELECT quantity
FROM Sales
WHERE quantity <> 5;
```

11. Arithmetic Operators

Arithmetic operators can be used to perform calculations directly in SQL queries.

10% Price Increase

```
SELECT
   product_name,
   price + price * 0.10 AS total_price
FROM Product;
```

This calculates the product price after adding a 10% increase.

Add 500 to Sale Amount

```
SELECT
   sale_amount,
   sale_amount + 500 AS new_sales_amount
FROM Sales;
```

12. Logical Operators

Logical operators allow multiple conditions to be combined.

Common logical operators:

- AND
- OR
- NOT

Price Between 5,000 and 50,000

```
SELECT price
FROM Product
WHERE price > 5000
AND price < 50000;
```

Quantity Equal to 2 or 4

```
SELECT sale_id, quantity
FROM Sales
WHERE quantity = 2
OR quantity = 4;
```

Price Not Greater Than 20,000

```
SELECT price
FROM Product
WHERE NOT price > 20000;
```

13. IS NULL and IS NOT NULL

IS NULL identifies missing values, while IS NOT NULL identifies values that are present.

Find NULL Product IDs

```
SELECT product_name, product_id
FROM Product
WHERE product_id IS NULL;
```

Since product_id is a primary key, this query returns no records.

Find Products with a Price

```
SELECT product_name, price
FROM Product
WHERE price IS NOT NULL;
```

14. IN and NOT IN

IN is used to match a value against multiple specified values.

Product IDs 1, 3, and 5

```
SELECT product_name, product_id
FROM Product
WHERE product_id IN (1, 3, 5);
```

Product IDs Not 2, 4, or 6

```
SELECT product_name, product_id
FROM Product
WHERE product_id NOT IN (2, 4, 6);
```

15. BETWEEN and NOT BETWEEN

BETWEEN is used to filter values within an inclusive range.

Price Between 1,000 and 20,000

```
SELECT product_name, price
FROM Product
WHERE price BETWEEN 1000 AND 20000;
```

Price Not Between 5,000 and 50,000

```
SELECT product_name, price
FROM Product
WHERE price NOT BETWEEN 5000 AND 50000;
```

16. LIKE Operator

The LIKE operator is used for pattern matching.

Product Names Starting with ‘M’

```
SELECT product_name
FROM Product
WHERE product_name LIKE 'M%';
```

Result includes:

Mouse
Monitor

Product Names Ending with ‘e’

```
SELECT product_name
FROM Product
WHERE product_name LIKE '%e';
```

Product Names Containing ‘phone’

```
SELECT product_name
FROM Product
WHERE product_name LIKE '%phone%';
```

This identifies:

Smartphone

17. NOT LIKE

NOT LIKE is used to exclude records matching a particular pattern.

Products Not Starting with ‘S’

```
SELECT product_name
FROM Product
WHERE product_name NOT LIKE 'S%';
```

18. Mixed / Exam-Level Queries

Products Between 1,000 and 20,000 Starting with ‘M’

```
SELECT product_name, price
FROM Product
WHERE price BETWEEN 1000 AND 20000
AND product_name LIKE 'M%';
```

This combines:

- BETWEEN
- AND
- LIKE

Sales with Quantity Between 2 and 10

```
SELECT sale_id, quantity
FROM Sales
WHERE quantity BETWEEN 2 AND 10;
```

Product ID in 1, 2, 3 and Price Greater Than 5,000

```
SELECT product_name, product_id
FROM Product
WHERE product_id IN (1, 2, 3)
AND price > 5000;
```

This combines IN and AND.

19. Key SQL Concepts Learned

Concept	SQL Syntax	Purpose
DISTINCT	DISTINCT	Retrieve unique values
Alias	AS	Rename output columns
Filtering	WHERE	Filter records
Greater Than	>	Compare values
Less Than	<	Compare values
Equal	=	Match values
Not Equal	<>	Exclude a value
AND	AND	Apply multiple conditions
OR	OR	Match either condition
NOT	NOT	Negate a condition
NULL Check	IS NULL	Find missing values
Non-NULL Check	IS NOT NULL	Find available values
Multiple Values	IN	Match multiple values
Exclusion	NOT IN	Exclude multiple values
Range	BETWEEN	Filter a range
Range Exclusion	NOT BETWEEN	Exclude a range

Concept	SQL Syntax	Purpose
Pattern Matching	LIKE	Search text patterns
Pattern Exclusion	NOT LIKE	Exclude text patterns
Calculation	+, -, *, /	Perform arithmetic

20. Skills Demonstrated

Through this project, the following SQL skills were practiced:

- MySQL Database Creation
- Table Creation
- Primary Keys
- Foreign Keys
- Constraints
- Data Insertion
- SELECT
- DISTINCT
- Column Aliasing
- WHERE
- Comparison Operators
- Arithmetic Operators
- Logical Operators
- IS NULL
- IS NOT NULL
- IN
- NOT IN
- BETWEEN
- NOT BETWEEN
- LIKE
- NOT LIKE
- Conditional Filtering
- Basic Data Analysis

21. Key Learning Outcomes

This coding challenge helped strengthen my understanding of how SQL is used to retrieve and filter data from relational databases.

I learned how to:

- Retrieve specific columns from a table.
- Display unique records.
- Rename columns using aliases.
- Filter data using different conditions.
- Perform calculations directly within SQL queries.
- Combine multiple conditions.
- Identify NULL and non-NULL values.
- Search for specific text patterns.
- Filter values within specific ranges.
- Write combined SQL conditions for analytical questions.

22. Project Results

The project successfully demonstrated the use of SQL clauses and operators on an e-commerce dataset.

The analysis included:

- Unique products and sale quantities.
- Products based on different price ranges.
- Sales based on quantity.
- Products matching specific IDs.
- Product names matching different text patterns.
- Calculated prices after a 10% increase.
- Modified sale amounts.
- Products satisfying multiple conditions.

23. Conclusion

The **SQL Coding Challenge – Clauses & Operators** provided practical experience in using fundamental SQL concepts with an e-commerce database.

By working with the Product and Sales tables, I gained a better understanding of how SQL can be used to **retrieve, filter, compare, calculate, and analyze data**.

These concepts form an important foundation for more advanced SQL topics such as **JOINs, GROUP BY, aggregate functions, subqueries, CTEs, window functions, and business data analysis**.

☆ Technologies

MySQL | SQL | MySQL Workbench